Contents

Volume 02 — Autoencoders (L10–L20) **1**
L10: Introduction to Autoencoder 1
L11: Reconstruction Loss (MSE, Binary Cross-Entropy) 2
L12: Types of Autoencoders 3
L13: Denoising Autoencoder 4
L14: Sparse Autoencoder 5
L15: Contractive Autoencoder 6
L16: Numerical Example and Limitations of Autoencoders 7
L17: Practical Exercise 1 — Autoencoders 8
L18: Practical exercise 2 — regularizing an autoencoder 9
L19: Intuition behind KL divergence — Part A 10
L20: Intuition behind KL divergence — Part B 11

Volume 02 — Autoencoders (L10–L20)

Generative AI & LLMs · NPTEL · Generative AI / LLMs **notes-3** — concept exam notes (definitions & what things do). Breadth over lab depth.

L10: Introduction to Autoencoder

Learning focus

- Place autoencoders in week 2 as **representation learning**, after week 1's activations, losses, optimizers, and CNNs.
- Contrast **supervised** $x \mapsto y$ mapping with **unsupervised** learning from unlabeled x only.
- Name the three AE blocks—**encoder**, **latent code**, **decoder**—and the two objectives: representation learning and reconstruction.
- Walk a fully connected encoder/decoder on tabular data: $100 \to 75 \to 25 \to 75 \to 100$.
- Distinguish **undercomplete** vs **overcomplete** latent spaces, including the identity-mapping failure mode.

Exam lines

- Autoencoders are unsupervised **representation learners** that reconstruct their own x; they are not $x \mapsto y$ classifiers.
- Encoder compresses x to latent h; decoder expands h to $\hat{x}$. Compression is **not** deletion of features.
- Two objectives: **representation learning** (encoder) and **reconstruction learning** (decoder).
- Tabular example: $100 \to 75 \to 25 \to 75 \to 100$, fully connected, sum + activation at each neuron. Latent size is a hyperparameter.
- **Undercomplete** ($\dim(h) < \dim(x)$) forces useful features. **Overcomplete** ($\dim(h) \geq \dim(x)$) risks copying x and padding zeros.

Concept tables

Block	Role
Encoder	Learns the most important features of x. Output is a **smaller internal representation**.
Latent code h	That compact representation (also: encoded data, latent representation, hidden representation).
Decoder	Maps h back toward the original input. Output is $\hat{x}$ (not called x, because it is reconstructed).
Objective	Who does it
———–	————-
Representation learning	Encoder
Reconstruction learning	Decoder

L11: Reconstruction Loss (MSE, Binary Cross-Entropy)

Learning focus

- Write the encoder/decoder maps $h = G(xW_E)$ and $\hat{x} = F(hW_D)$ with shapes for a 1000×6 example.
- Contrast **generic** weights (W_E and W_D both learned) with the **tied-weight** assumption $W_D = W^\top$.
- Choose **output activation** from the feature type: sigmoid for binary, linear for continuous.
- Derive **MSE** reconstruction loss for real-valued x and **binary cross-entropy** for $0/1$ features, including why BCE has a leading minus.
- Add a **weight regularizer** so W stays small; after training, $W^\star$ yields a good $h^\star$ and an $\hat{x}$ close to x.

Exam lines

- $h = G(xW_E)$, $\hat{x} = F(hW_D)$; for $1000 \times 6 \to 1000 \times 3$, W_E is 6×3 and W_D is 3×6.
- **Generic:** learn both matrices. **Tied:** learn W, decode with $W^\top$.
- Binary x: sigmoid out, **BCE**. Continuous x: linear out, **MSE**. Hidden: nonlinear (ReLU / sigmoid / tanh).
- Autoencoders have no class labels; reconstruction loss compares $\hat{x}$ to x.
- Regularize W (example $\lambda = 0.01$) so weights stay small; $W^\star$ gives a useful $h^\star$ and $\hat{x} \approx x$.

Concept tables

Object	Shape	Meaning
Input X	1000×6	1000 samples, 6 features
Latent H	1000×3	same 1000 samples, 3-D code (undercomplete)
Encoder weights W_E	6×3	maps $\mathbb{R}^6 \to \mathbb{R}^3$
Decoder weights W_D	3×6	maps $\mathbb{R}^3 \to \mathbb{R}^6$
Reconstruction $\hat{X}$	1000×6	6 features back
Setting	What is learned	Cost
———	———––	——
Generic	W_E **and** W_D	More parameters, slower training
Tied weights	Learn only W $(= W_E)$; set $W_D = W^\top$	Same shapes as needed for decode; **one** matrix to train

---

L12: Types of Autoencoders

Learning focus

- Build a **CNN autoencoder** for images: conv + max-pool encoder, upsample-then-conv decoder, ending in sigmoid for binary pixels.
- Compute the lecture's **28×28×1** Keras stack down to a **7×7×256** latent code and back.
- Contrast **nearest-neighbor / bed-of-nails** upsampling with **transposed convolution**, including the **checkerboard** artifact and why AE decoders prefer upsample + convolution.
- Code a **shallow** AE ($784 \to 32 \to 784$) and a **deep** AE ($784 \to 128 \to 64 \to 32 \to 64 \to 128 \to 784$).
- Explain why flattening a 28×28 image into 784 and using a deep MLP AE **loses spatial structure**.

Exam lines

- Image AE: encoder = conv + **max-pool**; decoder = **upsample then conv**; last layer **sigmoid** if pixels are $0/1$.
- Example stack: $28 \times 28 \times 1 \to \ldots \to$ latent $7 \times 7 \times 256 \to \ldots \to 28 \times 28 \times 1$.
- Nearest-neighbor / bed-of-nails upsample have **no** learned params. Transposed conv learns a kernel but can make **checkerboard** artifacts; prefer upsample + conv.
- Shallow: one hidden layer ($784 \to 32 \to 784$). Deep: stacked bottlenecks ($784 \to 128 \to 64 \to 32 \to \cdots \to 784$).
- Flattening images for an MLP AE loses spatial structure; use a CNN AE.

Concept tables

Stage	Operation	Output shape
Input	image	$28 \times 28 \times 1$
Conv2D	64 kernels, 3×3, ReLU, `padding='same'`	$28 \times 28 \times 64$
MaxPool	window 2×2	$14 \times 14 \times 64$
Conv2D	128 kernels, 3×3, ReLU, `same`	$14 \times 14 \times 128$
MaxPool	2×2	$7 \times 7 \times 128$
Conv2D	256 kernels, `same`	$7 \times 7 \times 256$ **(latent)**
Contribution	Where	Result (as taught)
——————–	——–	————————
G_1	1 at top-left	$1 \cdot [4, 3; 2, 1] \rightarrow 4, 3, 2, 1$ in the top-left 2×2
G_2	2 at top-right	$2 \cdot 4 = 8$, $2 \cdot 3 = 6$, $2 \cdot 2 = 4$, $2 \cdot 1 = 2$
G_3, G_4	remaining two inputs × kernel	two more 3×3 maps

L13: Denoising Autoencoder

Learning focus

- Say why AEs need **regularization**, especially in the **overcomplete** identity-mapping case.
- Draw the DAE pipeline: corrupt $x \rightarrow \tilde{x}$, encode, decode, compare $\hat{x}$ to the **clean** x (never to $\tilde{x}$).
- Name three corruption recipes: **Gaussian**, **masking** ($q = 0.4$), **salt-and-pepper**, with the lecture's numbers.
- Explain **neighboring-pixel dependence** as the reason a DAE can fill missing pixels (toy counts / means vs real **conv** filters).
- Write the DAE equations and training objective (MSE or BCE on clean x vs $\hat{x}$).

Exam lines

- Regularize AEs to stop identity mapping, especially when $\dim(h)$ is large.
- DAE: encode $\tilde{x}$, reconstruct **clean** x. That gap is the constraint.
- Corrupt with Gaussian (example $0.50 \rightarrow 0.51$, ...), masking ($q = 0.4$), or salt-and-pepper ($128 \rightarrow 0$, $130 \rightarrow 255$).
- Restoration uses **neighbor correlation**; toy fill is majority/mean; real images use **conv** kernels (e.g. 32 filters, 3×3).
- Train MSE or BCE on $(x, \hat{x})$; test: noisy in, clean out.

Concept tables

	x_1	x_2	x_3	x_4
Clean x	0.50	0.80	0.20	0.90
After noise $\tilde{x}$	0.51	0.75	0.23	0.82

Corruption	What it does
Gaussian	Add $\mathcal{N}(0, \sigma^2)$
Masking	Each feature $\to 0$ with probability q (lecture $q = 0.4$)
Salt-and-pepper	Random pixels $\to$ extreme intensity (0 or 1 binary; 0 or 255 grayscale)

L14: Sparse Autoencoder

Learning focus

- State the SAE idea: even with a **wide** hidden layer, keep **most units inactive** so the few that fire must carry important features.
- Write the loss as reconstruction **plus** a sparsity penalty controlled by β.
- Define average hidden activation $\hat{\rho}_j$, desired sparsity ρ, and the constraint $\hat{\rho}_j \approx \rho$.
- Use **KL divergence** $\mathrm{KL}(\rho \| \hat{\rho}_j)$ as the penalty, including the active and inactive terms, and the identity case $\rho = \hat{\rho} = 0.05$.
- Follow the lecture's **5-sample / 7-neuron** example: large $\hat{\rho}$ on **H2** and **H6** → large KL → backprop shrinks those units.

Exam lines

- SAE: many hidden units allowed, but **most stay off**; the few that fire must be useful.
- Loss = reconstruction + $\beta\times$ sparsity penalty (KL in this lecture; L1 is the other option).
- $\hat{\rho}_j$ = mean activation of unit j over m samples; target ρ; want $\hat{\rho}_j \approx \rho$.
- $\mathrm{KL}(\rho \| \hat{\rho})$ has an **active** log and an **inactive** log; $\rho = \hat{\rho} = 0.05$ gives $\mathrm{KL} = 0$.
- In the 5×7 example, **H2** and **H6** are over-active; large KL + backprop makes them quieter.

Concept tables

Term	Role
$L_{\text{reconstruction}}$	How well $\hat{x}$ matches x
L_{sparsity}	**Penalty** if too many hidden neurons are active
β	Hyperparameter: **how strongly** to enforce sparsity (small constant)
Term	What it matches
——	——————–
$\rho \log(\rho/\hat{\rho}_j)$	**Active** part: how often the unit **should** fire vs how often it **does**
$(1-\rho)\log\big((1-\rho)/(1-\hat{\rho}_j)\big)$	**Inactive** part: how often it **should** be off vs how often it **is**

L15: Contractive Autoencoder

Learning focus

- State the CAE extra requirement: reconstruct well **and** keep h **stable** when x changes only slightly (noise, brightness).
- Write the **Jacobian** $J_h(x) = \partial h/\partial x$ and the **Frobenius** penalty $\|J_h(x)\|_F^2$.
- Add that penalty to reconstruction loss with weight λ.
- Interpret a large vs small $\partial h_1/\partial x_1$ (sensitivity vs insensitivity).
- Explain the **trade-off**: keep **genuine** variations, suppress **unimportant** ones (noise), using λ and the two loss terms.

Exam lines

- CAE: $\hat{x}$ should match x, and $h(x)$ should match $h(x + \text{small noise})$.
- Penalty is λ times the **Frobenius** norm of $J_h(x) = \partial h/\partial x$.
- Large Jacobian entries = h too sensitive (example $x = (1.5, 2.5, 3.5)$ vs $(1.6, 2.5, 3.4)$).
- L_{rec} vs Jacobian is a trade-off: keep **real** factors, drop **noise**; λ sets the balance.
- Overcomplete identity mapping is the failure mode this regularizer is built to block.

Concept tables

Size of $\partial h_1/\partial x_1$	Meaning	Wanted?
Large	h_1 changes a lot when x_1 changes a little; h_1 is **very sensitive** to x_1	No: reconstruction would **copy noise** into $\hat{x}$

Size of $\partial h_1/\partial x_1$	Meaning	Wanted?
Small	h_1 barely reacts; **insensitive** to small x_1 wiggles	Yes for **unimportant** wiggles
λ	Effect (as taught)	
———–	—————————	
Large	Punish h-movement more → **more stable** features	
Small	Weaker penalty → **more detailed** reconstruction	

L16: Numerical Example and Limitations of Autoencoders

Learning focus

- Run a fully connected AE **forward pass** with the lecture's numbers: $5 \to 4 \to 3 \to 4 \to 5$.
- Use **ReLU** on hidden layers and **linear + MSE** on a continuous output.
- State what training does to weights/biases so $\hat{x}$ approaches x.
- List four **limitations** of AEs (uninterpretable h, weak downstream use, approximation not generation, latent-size trap).
- See why week 3 adds a **VAE** (KL regularizer on the latent space).

Exam lines

- Worked net: $x = (0.8, 0.4, 0.6, 0.2, 0.5)$, $5 \to 4 \to 3 \to 4 \to 5$, ReLU hidden, linear out.
- Spoken checkpoints: $H_1 = (0.72, 0.65, 0.66, 0.77)$, $h = (0.760, 0.714, 0.944)$, $\hat{x}_1 = 0.67838$; then MSE and backprop.
- Low reconstruction error does **not** imply a meaningful or reusable h, and AEs **reconstruct** rather than sample new x.
- Tiny latent loses information; huge latent copies. VAEs add **KL on** h to fix the latent-space issues.

Concept tables

	x_1	x_2	x_3	x_4	x_5
Original x	0.8	0.4	0.6	0.2	0.5
$\hat{x}$ (first coord. spoken)	0.67838	(slide)	(slide)	(slide)	(slide)
x type	Hidden act.	Output act.	Loss		
———	————	————	——		
Continuous (this example)	ReLU	**Linear**	**MSE**		
Binary $0/1$	ReLU	**Sigmoid**	**BCE**		

L17: Practical Exercise 1 — Autoencoders

Learning focus

- Load MNIST **without labels**, scale to $[0, 1]$, flatten $28 \times 28 \to 784$.
- Train a **shallow** AE $784 \to 64 \to 784$ (Adam, BCE) and read the parameter counts $50{,}240 + 50{,}960 = 101{,}200$.
- Train a **deep** AE $784 \to 256 \to 128 \to 64 \to 128 \to 256 \to 784$ and compare validation loss to the shallow net.
- Add **L1** (10^{-4}) for a **sparse** AE and see higher loss / worse reconstructions.
- Build a **DAE**: Gaussian noise with `noise_factor=0.5`, clip to $[0, 1]$, train on noisy inputs vs clean targets.

Exam lines

- Drop MNIST labels; reconstruct `x` from `x` (or from `x_noisy` in the DAE).
- Shallow 64-D bottleneck: **101,200** weights; val. loss down to ~ 0.09 in 5 epochs.
- Deep has **484,944** weights; here it does not beat shallow on val. loss in 5 epochs.
- Sparse L1 (10^{-4}) **raises** loss (~ 0.43) and hurts visual reconstructions in this run.
- DAE: `noise_factor=0.5` Gaussian, clip $[0, 1]$, train `(noisy, clean)`; recovers a clean digit from a noisy one.

Concept tables

Import	Role
`numpy` as `np`	Arrays / numbers
`matplotlib.pyplot` as `plt`	Plots
`tensorflow` as `tf`	Deep-learning framework
`mnist` from `tensorflow.keras.datasets`	Handwritten digits **0–9** (10 classes in the dataset, unused as labels)
`Dense`, `Input`	Hidden/output dense layers; input size
`Model`	Functional API model
Split	Count
——-	——:
Train	60,000
Test	10,000

L18: Practical exercise 2 — regularizing an autoencoder

Learning focus

- Implement five regularization methods on the same MNIST autoencoder: **L1**, **L2**, **dropout**, **batch normalization**, and **early stopping**.
- State what each method does to weights or connections (absolute vs squared penalty, dropped edges, per-batch scaling, patience).
- Train with the lecture's optimizer, loss, epochs, and batch size, and compare **validation losses**.
- Read the lecture's ranking: L2 beats L1; among all five runs, **early stopping** is called the best.

Exam lines

- Same data recipe every time: MNIST, float, $/255$, shape `(-1, 784)`, **Adam + BCE**, reconstruct `x` from `x`, **5 × batch 256**.
- L1 taxes $|w|$; L2 taxes w^2; both at $\lambda = 10^{-4}$ on a **128**-unit bottleneck. L2's last val. loss **0.09** beats L1's **0.43**.
- Dropout **0.3** after 256 and after 128 stops co-adaptation so test reconstructions generalize.
- Batch norm uses $(x - \mu)/\sigma$ and reached **0.08** in this run.
- Early stopping **monitors `val_loss`**, **patience 2**, **restores best weights**; here loss still fell to **0.0722** in five epochs, so the callback never clipped training.

Concept tables

Import	Role in the notebook
`numpy` as `np`	Arrays
`matplotlib.pyplot` as `plt`	Plots (available; this session is mostly loss numbers)

Import	Role in the notebook
`tensorflow`	Neural nets
`tensorflow.keras.datasets.mnist`	Handwritten digits **0–9** (10 classes)
* from TensorFlow / Keras	Pull in layers and model helpers in one go
Keras `models`	Build the autoencoder
Regularizers **L1, L2**; **Dropout**; **BatchNormalization**; **EarlyStopping**	The five techniques

Choice	Value
——–	——-
Input size	784
Output activation	**Sigmoid**
Optimizer	**Adam**
Loss	**Binary cross-entropy**
Fit targets	`(x_train, x_train)`
Epochs	**5**
Batch size	**256**
Validation	`(x_test, x_test)`

L19: Intuition behind KL divergence — Part A

Learning focus

- Say why **KL divergence** exists: Euclidean distance is for **points**; KL is for **probability distributions** $P(x)$ vs $Q(x)$ (discrete or continuous).
- Read the three overlap pictures: identical → KL $= 0$; nearby means → small KL; far apart → large KL.
- Write **forward** $D_{\mathrm{KL}}(P \,\|\, Q)$ and **reverse** $D_{\mathrm{KL}}(Q \,\|\, P)$ for sums and integrals.
- Match **forward = mode covering** and **reverse = mode seeking** on the height (160 / 190 cm) and cats-vs-dogs examples.

Exam lines

- KL replaces Euclidean distance when P and Q are **densities**, discrete or continuous.
- Identical densities → KL **0**; slight mismatch → **small**; far means → **large**.
- **Forward** KL makes Q **cover** P (risk: gap samples). **Reverse** KL makes Q **seek one mode** (risk: drop the other mode).
- In a VAE, P is the unknown data distribution and Q_θ is the learned approximation; KL is how we **score** the mismatch. The algebraic identity is the **next** lecture.

Concept tables

Topic this week	Where
Intuition behind KL	This lecture + Part B
VAE architecture	Encoder, **latent space**, decoder (later videos)
ELBO	Evidence lower bound (later)
VAE loss	**Reconstruction + KL**
Reparameterization trick	Later
Numerical example	End of the week
Case	Overlap
——	———
P and Q **identical** (blue P, red Q stacked)	Complete
Nearby means, **good but not perfect** overlap	Partial
Means far, **almost no overlap**	Little

L20: Intuition behind KL divergence — Part B

Learning focus

- Define **self-information** $I(x) = -\log P(x)$ as surprise of **one** outcome, with the weather numbers in **bits**.
- Define **entropy** $H(P)$ as the **average** surprise of a whole distribution (one number for uncertainty).
- Define **cross-entropy** $H(P, Q)$ as surprise of the **true** outcome under the **model's** Q.
- Derive **forward KL** as the lecture did: $D_{\mathrm{KL}}(P \,\|\, Q) = H(P, Q) - H(P)$, down to $\sum P \log(P/Q)$.

Exam lines

- $I(x) = -\log P(x)$: surprise of **one** outcome (sunny **0.193 bits**, rain **3 bits** at these P).
- $H(P)$: average surprise of **all** outcomes (**0.544** here; **1** if 50–50; **0** if deterministic).
- $H(P, Q)$: score Q against true P (small if Q is high on the correct class).
- Forward KL is **cross-entropy minus entropy**, which algebraically is $\sum P \log(P/Q)$.

Concept tables

Outcome x	$P(x)$	Spoken percent
Sunny	**0.875**	87.5% (also said "around 87% / 90%")
Rainy	**0.125**	12.5%
$P(x)$	$I(x)$	Reading
——–	——–	———
Large (sunny happened, as predicted)	**Small**	Little surprise
Small (rain happened)	**Large**	Much surprise
